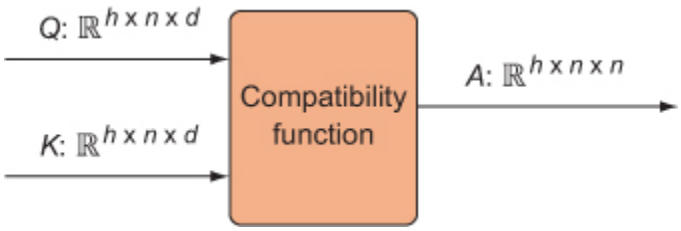
dimension is equal to the output channel dimension of the last convolutional layer plus the two spatial coordinates that we append. The linear layer thus contracts over the channel dimension to give us query, key, and value matrices, $Q,K,V:\mathbb{R}^{b\times n\times(h\times d)}$, so we will use the Einops `rearrange` function to expand out the last dimension into head and *d* dimensions.

```
>>> K = rearrange(self.k_proj(x), "b n (head d) -> b head n d",
head=self.n_heads)
```

We will extract out the separate head and *d* dimension and simultaneously reorder the dimensions so that the head dimension comes after the batch dimension. Without Einops, this would be more code and not nearly as readable.

For this example, we will also abandon the dot (inner) product as the compatibility function (recall, this is the function that determines the similarity between the query and keys) and instead use something called *additive attention* (figure 10.20). The dot product attention would work fine, but we wanted to illustrate that it is not the only kind of compatibility function, and the additive function is actually a bit more stable and expressive.

Figure 10.20. The compatibility function computes the similarity between each key and query vector, resulting in an adjacency matrix.



With dot product attention, we compute the compatibility (i.e., the similarity) between each query and key by simply taking the dot product between each vector. When the two vectors are similar element-wise, the dot product will yield a large positive value, and when they are dissimilar, it may yield a value near zero or a big negative value. This means the output of the (dot product) compatibility function is unbounded in both directions, and we can get arbitrarily large or small values. This can be problematic